Studio Nx

Backend(FastAPI/Python) 89 — DevNoteAdmin.delete_model() 소프트 삭제

2026-10-12

Backend(FastAPI/Python) 89 — `DevNoteAdmin.delete_model()` (소프트 삭제)

항목	내용
파일	`studionx/backend/app/admin.py`
작업	P1-21 공지 API와 관리자 화면: 삭제 버튼이 행을 지우지 않고 `deleted_at`만 채우게 하기 (PRD 01 FR-115, `DB.md` 소프트 삭제 규칙)
작성	Claude (AI), 2026-10-13, 사용자 검토
이전 · 다음	88 `DevNoteAdmin` · 90 `InquiryAdmin` (다음)

1. 하는 일

관리자 화면에서 공지의 삭제 버튼을 누르면, 행을 지우는 대신 **`deleted_at`에 지금 시각(UTC)을 넣습니다.** `DB.md`가 `dev_notes`를 소프트 삭제 대상으로 정했기 때문입니다.

상황	결과
공지를 삭제함	`deleted_at`이 채워지고 행은 DB에 남습니다
삭제한 공지	관리자 목록, 상세, 수정 화면(88번)과 메인 페이지 공지 API(83, 84번)에서 모두 사라집니다
이미 지운 공지를 다시 삭제	아무것도 바꾸지 않습니다 (처음 지운 시각이 남음)
없는 번호를 삭제	오류 없이 끝납니다
되살리기	DB에서 `deleted_at`을 비우면 목록에 다시 나타납니다

2. 언제 쓰나

호출하는 곳	화면
SQLAdmin의 삭제 요청 (`/admin/dev-note/delete`)	공지 목록의 삭제 버튼, 여러 개를 골라 지우기

3. 코드

import (이번에 추가한 것)

```
from starlette.concurrency import run_in_threadpool
```

함수

```
async def delete_model(self, request: Request, pk) -> None:
    await run_in_threadpool(self._soft_delete, pk)

def _soft_delete(self, pk) -> None:
    with self.session_maker() as session:
        note = session.get(DevNote, int(pk))
        if note is not None and note.deleted_at is None:
```

```
            note.deleted_at = datetime.now(UTC).replace(tzinfo=None)
            session.commit()
```

백엔드(Python)는 지금 방식 그대로여서 "최신 문법" 버전은 따로 두지 않습니다.

문법 풀이

식	뜻
`delete_model(self, request, pk)`	SQLAdmin이 삭제할 때 부르는 함수입니다. 기본 동작(`session.delete`)을 바꿔 덮어씁니다
`run_in_threadpool(함수, 값)`	일반(동기) 함수를 서버가 멈추지 않도록 별도 스레드에서 실행하고 기다립니다. 이 프로젝트의 DB 연결은 동기 방식이기 때문입니다
`with self.session_maker() as session`	SQLAdmin이 `Admin(...)`을 만들 때 넣어 주는 DB 세션 공장입니다. 블록이 끝나면 세션을 닫습니다
`session.get(DevNote, int(pk))`	기본키로 공지 하나를 가져옵니다. `pk`는 글자로 오므로 정수로 바꿉니다
`if note is not None and note.deleted_at is None`	있고 아직 안 지운 공지일 때만 바꿉니다

4. 흐름

1. SQLAdmin이 삭제 요청을 받고, 먼저 그 공지가 있는지 확인한 뒤 `delete_model`을 부릅니다.
2. 공지를 찾아서 아직 안 지웠으면 `deleted_at`에 지금 시각을 넣고 저장합니다.
3. SQLAdmin이 목록으로 돌아가며, 이미 `deleted_at`이 있어서 그 공지는 보이지 않습니다.

5. 왜 이렇게

선택	이유
행을 지우지 않음	실수로 지워도 `deleted_at`을 비우면 되살릴 수 있고, `DB.md`의 소프트 삭제 규칙을 따릅니다
이미 지운 공지는 건드리지 않음	처음 지운 시각이 기록으로 남습니다. 같은 요청이 두 번 와도 결과가 같습니다
없는 번호는 오류 없이	삭제는 "그 상태가 되면 됨"이어서, 이미 없으면 목적이 달성된 것입니다
`can_delete = False`로 버튼을 없애지 않음	관리자에게는 "삭제" 버튼이 자연스럽고, 안에서만 소프트 삭제로 바꿉니다
동기 세션을 스레드에서 실행	`async` 함수 안에서 동기 DB를 직접 부르면 그동안 서버가 멈춥니다. SQLAdmin 자신도 같은 방식으로 합니다
이 함수에서 `on_model_delete` 같은 후크를 부르지 않음	지금 쓰는 후크가 없습니다

6. 시험 (`tests/test_admin_dev_note.py`, 이번에 5개 추가)

시험	결과
삭제하면 `deleted_at`이 채워지고(UTC, 시간대 없음) 행은 그대로 있음	통과입니다
공개된 공지를 지우면 메인 페이지 서비스(83 목록, 84 상세)에서 사라지고, 상세는 404	통과입니다
두 번 지워도 처음 지운 시각이 그대로	통과입니다

시험	결과
없는 번호(`999999999`)를 지워도 오류 없음	통과입니다
`deleted_at`을 비우면 다시 목록에 나타남 (되살리기)	통과입니다

이 파일의 시험은 88번의 9개와 합쳐 14개이며 모두 통과했습니다. `ruff`도 통과했습니다. SQLAdmin 화면의 삭제 버튼을 실제로 누르는 확인은 `setup_admin()`(92번) 이후 서버를 띄워서 합니다.